Two-Body Gravity

A real-time gravitational simulation in Rust

Abstract

A real-time simulation of two point masses under mutual Newtonian gravity. The physics runs in double precision on a fixed timestep, integrated with velocity Verlet; rendering is immediate-mode OpenGL via macroquad. This document derives every formula the program uses, explains why each numerical method was chosen over its alternatives, and describes how the results are verified.

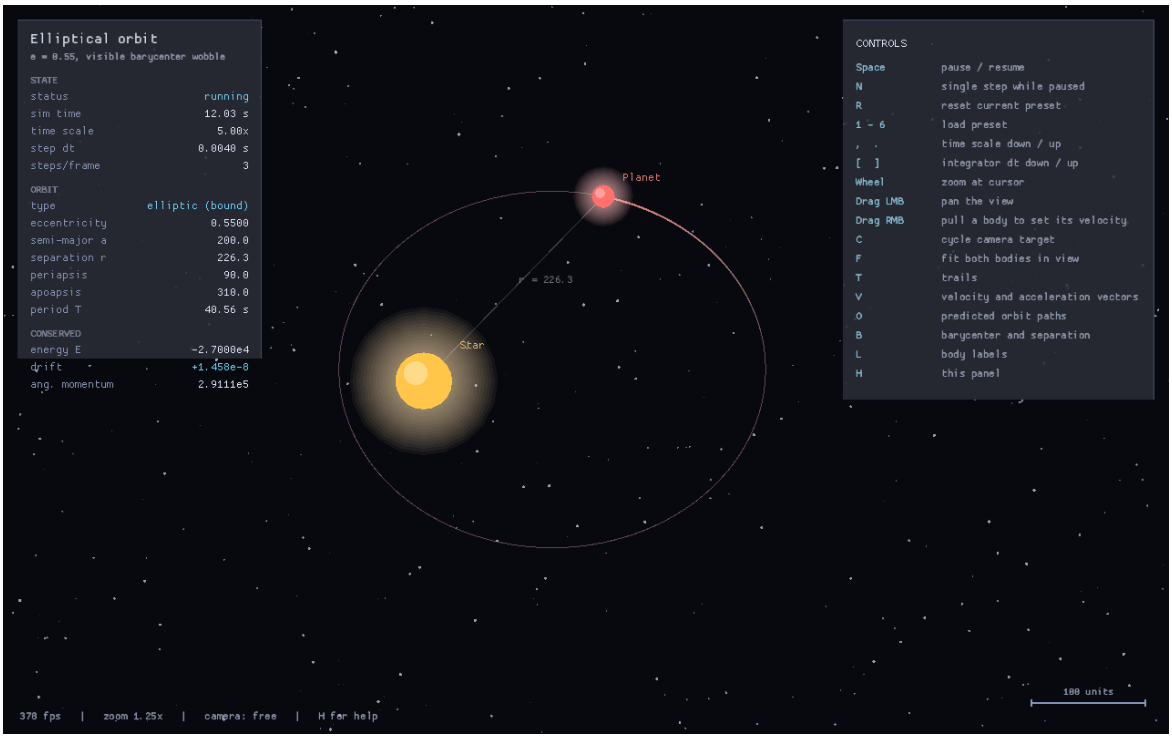


Figure 1: The elliptical preset ($e = 0.55$). The panel on the left reports orbital elements derived analytically from the instantaneous state vector, independently of the integrator that produced it.

Contents

1 Running the program **2**

1.1 Controls 2

1.2 Presets 2

2 The physical model **3**
2.1 Deriving the acceleration . . . 3
2.2 The cube is not an inverse-cube force . . . 3
2.3 Why m_0 cancels . . . 4
2.4 Why the state is stored as vectors . . . 4

3 Softening **5**

4 Integration: velocity Verlet **5**
4.1 Why this integrator and not RK4 . . . 5
4.2 The drift readout . . . 6

5 Decoupling physics from the frame rate **6**

6 Conserved quantities **7**

7 Reduction to relative motion **7**

8 Orbital elements from the state vector **7**
8.1 The eccentricity vector . . . 8
8.2 Classification is by energy, not eccentricity . . . 8

9 Drawing the predicted orbit **8**

10 Building the initial conditions **9**

11 Units **9**

12 Rendering **9**

13 Source layout **10**

14 Tests **10**

15 Building this document **11**
15.1 Prerequisites . . . 11
15.2 Generating the PDF . . . 11

1 Running the program

The simulation is a standard Cargo project and needs only a Rust toolchain. Install one from https://rustup.rs if you do not have it, then:

```
cargo run --release
```

The `-release` flag matters: the debug build is roughly an order of magnitude slower.

On Linux, macroquad additionally needs the system OpenGL and X11 development libraries. On Debian or Ubuntu:

```
sudo apt install libx11-dev libxi-dev libgl1-mesa-dev
```

Two optional flags are accepted:

Flag	Effect
`-preset <1-6>`	start on a given configuration
`-capture <path>`	render a few seconds, write a PNG, exit

The second exists so the visuals can be checked without a person at the keyboard, which is how the figures in this document were produced.

1.1 Controls

Key	Action
`Space`	pause / resume
`N`	single step while paused
`R`	reset the current preset
`1`–`6`	load a preset
`, .`	time scale down / up
`[ ]`	integrator step size down / up
Wheel	zoom at the cursor
Drag LMB	pan
Drag RMB	pull from a body to set its velocity
`C`	cycle camera target (free / barycenter / either body)
`F`	frame the whole orbit
`T V O B L`	toggle trails, vectors, predicted orbits, barycenter, labels
`P`	save a screenshot
`H`	controls panel
`Esc` / `Q`	quit

1.2 Presets

1. **Circular orbit** — $e = 0$, light satellite.
2. **Elliptical orbit** — $e = 0.55$, with a visible barycenter wobble.
3. **Comet** — $e = 0.9$, a fast periapsis passage.
4. **Equal-mass binary** — two identical bodies on mirrored ellipses.
5. **Hyperbolic flyby** — unbound; deflected once and gone.
6. **Radial infall** — zero angular momentum, straight-line collapse.

2 The physical model

Two point masses m_0 and m_1 at positions $\mathbf{r}_0$ and $\mathbf{r}_1$. Everything below is built from the separation vector between them:

$$\mathbf{d} = \mathbf{r}_1 - \mathbf{r}_0.$$

2.1 Deriving the acceleration

Step 1 — Newton's law of gravitation, in the form everyone learns it. Note that it gives only a *magnitude*:

$$|\mathbf{F}| = \frac{G\, m_0\, m_1}{r^2}, \qquad r = |\mathbf{d}|.$$

Step 2 — attach a direction. Force is a vector, so the magnitude needs a unit vector to point it. Body 0 is pulled *toward* body 1, and since $\mathbf{d}$ points from 0 to 1, that direction is $\hat{\mathbf{d}} = \mathbf{d}/|\mathbf{d}|$:

$$\mathbf{F}_0 \;=\; \frac{G\, m_0\, m_1}{|\mathbf{d}|^2}\,\hat{\mathbf{d}} \;=\; \frac{G\, m_0\, m_1}{|\mathbf{d}|^2}\cdot\frac{\mathbf{d}}{|\mathbf{d}|} \;=\; \frac{G\, m_0\, m_1\, \mathbf{d}}{|\mathbf{d}|^3}.$$

Step 3 — Newton's second law, $\mathbf{F} = m\mathbf{a}$, so divide through by m_0:

$$\mathbf{a}_0 = \frac{G\, m_1\, \mathbf{d}}{|\mathbf{d}|^3}, \tag{1}$$

$$\mathbf{a}_1 = -\frac{G\, m_0\, \mathbf{d}}{|\mathbf{d}|^3}. \tag{2}$$

Equations (1) and (2) are the entire physics. The rest of the simulation is machinery for integrating them accurately and drawing the result.

2.2 The cube is not an inverse-cube force

Gravity here is still $1/r^2$. The exponent reads 3 only because $\mathbf{d}$ in the numerator is *unnormalized* — it carries a factor of $|\mathbf{d}|$ in its own length. Splitting it back apart:

$$\frac{\mathbf{d}}{|\mathbf{d}|^3} \;=\; \underbrace{\frac{1}{|\mathbf{d}|^2}}_{\text{magnitude}} \cdot \underbrace{\hat{\mathbf{d}}}_{\text{direction}}$$

Fusing the two is a numerical convenience: normalizing $\mathbf{d}$ first and then scaling would need a second division by $|\mathbf{d}|$. The code computes $|\mathbf{d}|^3$ as `r2 * r2.sqrt()`, that is $|\mathbf{d}|^2 \cdot |\mathbf{d}|$, for one square root in total.

Dimensions confirm it is an acceleration. With $[G] = \mathrm{m^3\,kg^{-1}\,s^{-2}}$:

$$\frac{\mathrm{m^3\,kg^{-1}\,s^{-2}\cdot kg\cdot m}}{\mathrm{m^3}} \;=\; \mathrm{m\,s^{-2}}.$$

Leaving $|\mathbf{d}|^2$ in the denominator would yield $\mathrm{m^2\,s^{-2}}$, which is not an acceleration — the cube is what makes the units come out right.

2.3 Why m_0 cancels

The *force* on body 0 depends on both masses, but its *acceleration* depends only on m_1, the mass of the other body. The m_0 in the gravitational law is cancelled by the m_0 in $\mathbf{F} = m_0\mathbf{a}$.

That cancellation is the equivalence principle — how strongly a body is pulled and how strongly it resists being pulled are the same number, so heavy and light objects fall identically. It is visible directly in `System::compute_accelerations`, where each body's acceleration is scaled by the *other* body's mass:

```
self.bodies[0].acc = d * (self.g * m1 * inv_r3);   // uses the OTHER mass
self.bodies[1].acc = d * (-self.g * m0 * inv_r3);
```

The only difference between the two lines is the sign. Both bodies accelerate along the same axis $\mathbf{d}$ in opposite directions, which is Newton's third law, and why the pair's total momentum never changes.

2.4 Why the state is stored as vectors

A two-body orbit is planar, so it could be described with scalars — a radius and an angle, or a set of orbital elements — and some simulations do exactly that. This one keeps position, velocity and acceleration as `V2` vectors and integrates the ODE directly. The reasons are concrete:

- **A magnitude alone is not a force.** Gm_0m_1/r^2 says how hard, never which way. The direction has to be reintroduced somehow; $\mathbf{d}/|\mathbf{d}|^3$ carries both at once and keeps the expression in a single term.
- **No trigonometry in the inner loop.** A polar formulation needs atan2, sin and cos every step. The vector form needs one square root and a handful of multiplies, and it never has to unwrap an angle across the 2π boundary.
- **No special cases at the degenerate configurations.** In polar coordinates a radial free-fall has $\dot{r}$ finite but $\dot{\theta}$ undefined, and angular formulations need a branch to survive it. Preset 6 does exactly this, and the vector code runs it with no special handling: $\mathbf{d}$ simply shrinks along a fixed direction. (The separate problem of $|\mathbf{d}| \to 0$ blowing up the force is what softening in Section 3 addresses; that one is real, and no choice of coordinates avoids it.)
- **The orbit's orientation is an output, not a coordinate choice.** Nothing in the integrator knows where periapsis is or which way the orbit is tilted. The argument of periapsis ω is *derived* in Section 8 from the state, which is what makes those elements an independent check on the integrator rather than an assumption fed into it.
- **Forces superpose as vectors.** Adding a third body means summing $\mathbf{d}/|\mathbf{d}|^3$ terms over pairs; the loop body does not change. Scalar magnitudes cannot be summed without reintroducing direction anyway.
- **The integrator is agnostic.** Velocity Verlet (Section 4) is written once in terms of + and multiplication by a scalar, and applies unchanged to position, velocity and acceleration because all three are the same type. That is why `src/vec2.rs` implements the operator traits — the physics reads like the equations above.

The cost is that `V2` is a hand-rolled `f64` type rather than the `f32` `Vec2` macroquad provides. Orbits accumulate error over millions of steps, and `f32`'s ∼7 decimal digits are not enough.

The narrowing to `f32` happens only at the drawing boundary, in `Camera::world_to_screen` — so precision is spent on the trajectory and merely rounded off on the way to a pixel.

The whole system state is six numbers per body: position, velocity, and the cached acceleration. There is no force accumulator, no collision resolution, and no drag.

3 Softening

A literal $1/r^2$ force diverges as the separation goes to zero. The radial-infall preset drives the bodies straight through each other, so this is not hypothetical. The fix is **Plummer softening**: replace the distance with a value that has a floor,

$$|\mathbf{d}|^3 \;\longrightarrow\; \left(|\mathbf{d}|^2 + \varepsilon^2\right)^{3/2},$$

with $\varepsilon = 0.5$ world units by default. The important property is that this softened force is still the exact gradient of a softened *potential*,

$$U = -\frac{G\, m_0\, m_1}{\sqrt{|\mathbf{d}|^2 + \varepsilon^2}}, \qquad \mathbf{F}_0 = -\nabla_{\mathbf{r}_0} U,$$

so the softened system remains perfectly Hamiltonian and energy conservation stays meaningful. The simulation is exactly integrating a slightly different, non-singular force law rather than approximating the real one badly.

This consistency is not optional. Soften the force but not the potential and the force stops being any potential's gradient, at which point the conserved energy reported in Section 6 would be measuring nothing.

At the separations the presets use the correction is negligible. For $r = 220$ and $\varepsilon = 0.5$,

$$\left(1 + \frac{\varepsilon^2}{r^2}\right)^{3/2} = 1 + 7.7 \times 10^{-6},$$

under ten parts per million.

4 Integration: velocity Verlet

Given accelerations, the integrator advances the state by Δt:

$$\mathbf{r}(t + \Delta t) = \mathbf{r}(t) + \mathbf{v}(t)\,\Delta t + \tfrac{1}{2}\,\mathbf{a}(t)\,\Delta t^2 \qquad \text{drift} \tag{3}$$

$$\mathbf{a}(t + \Delta t) = \mathbf{a}\big(\mathbf{r}(t + \Delta t)\big) \qquad \text{recompute forces} \tag{4}$$

$$\mathbf{v}(t + \Delta t) = \mathbf{v}(t) + \tfrac{1}{2}\big[\mathbf{a}(t) + \mathbf{a}(t + \Delta t)\big]\Delta t \qquad \text{kick} \tag{5}$$

The drift step (3) uses the *old* acceleration; the kick step (5) averages old and new. That averaging is the entire trick, and it is why the old acceleration is cached on each `Body` rather than recomputed. The cost is one force evaluation per step.

4.1 Why this integrator and not RK4

Velocity Verlet is *symplectic*: it exactly conserves a "shadow" Hamiltonian that differs from the true one by $O(\Delta t^2)$. The practical consequence is that its energy error is **bounded and oscillatory** rather than accumulating.

Integrator	Order	Energy behaviour
Explicit Euler	1	pumps energy in; the orbit spirals outward
RK4	4	very accurate per step, but slowly bleeds energy
Velocity Verlet	2	error oscillates in a fixed band; the orbit closes

RK4 has the better local truncation error and the worse long-run behaviour. For a simulation meant to run for millions of steps, bounded error beats small error. Verlet is also time-reversible: stepping forward then backward by Δt returns to the starting state up to rounding.

4.2 The drift readout

The heads-up display reports

$$\text{drift} = \frac{E - E_0}{|E_0|},$$

which is the live evidence for the paragraph above. It reads $+5.4 \times 10^{-15}$ on the circular preset — the `f64` noise floor — and between 10^{-9} and 10^{-8} on the eccentric ones, and it does not trend upward over time. A growing value means Δt is too large for the current periapsis speed; `[` shrinks it.

5 Decoupling physics from the frame rate

Stepping by the frame time would make the trajectory depend on rendering speed, so results would differ between machines and change when the window is dragged. Instead the simulation banks elapsed real time and spends it in fixed Δt chunks:

```
accumulator += min(frame_time, 0.05) * time_scale;
while accumulator >= dt && substeps < 6000 {
    step(dt);
    accumulator -= dt;
}
```

Three details matter:

- `min(frame_time, 0.05)` — clamping the frame delta stops a stall (window drag, first frame, a breakpoint) from queuing an unbounded backlog of physics.
- `substeps < 6000` — capping the inner loop makes an extreme time scale degrade into slow motion instead of freezing the window.
- `time_scale` multiplies simulated time per real second; it never touches Δt, so speeding up the view cannot change the trajectory. Only `[` and `]` change Δt, and those are the accuracy control.

Defaults are $\Delta t = 0.004\,\text{s}$ and a time scale of 5.

6 Conserved quantities

The panel tracks the quantities the true dynamics must preserve. They are computed from the raw state, independently of the integrator, so they act as a running self-check.

$M = m_0 + m_1$	total mass
$\mathbf{R} = \dfrac{m_0\mathbf{r}_0 + m_1\mathbf{r}_1}{M}$	barycenter
$\mathbf{P} = m_0\mathbf{v}_0 + m_1\mathbf{v}_1$	linear momentum
$K = \sum_i \frac{1}{2} m_i \lvert\mathbf{v}_i\rvert^2$	kinetic energy
$U = -\dfrac{G m_0 m_1}{\sqrt{\lvert\mathbf{d}\rvert^2 + \varepsilon^2}}$	potential energy
$E = K + U$	total energy
$L = \sum_i m_i \, (\mathbf{r}_i - \mathbf{R}) \times (\mathbf{v}_i - \mathbf{V})$	angular momentum

L is taken about the barycenter and in the barycenter's own velocity frame ($\mathbf{V} = \mathbf{P}/M$), so it stays invariant even if the pair is drifting. In two dimensions the cross product is the scalar $x_1 y_2 - y_1 x_2$.

Every preset is passed through `zero_momentum()` at load, which subtracts $\mathbf{R}$ from the positions and $\mathbf{V}$ from the velocities. This puts the pair in the center-of-mass frame with $\mathbf{P} = 0$ and $\mathbf{R} = 0$, so the barycenter is a fixed point and the orbits stay on screen. Setting a velocity by hand with right-drag breaks that, which is why the camera can follow the barycenter.

7 Reduction to relative motion

Subtracting the two accelerations of Section 2 collapses the pair into a single body:

$$\ddot{\mathbf{d}} = \mathbf{a}_1 - \mathbf{a}_0 = -\frac{G(m_0 + m_1)\,\mathbf{d}}{\lvert\mathbf{d}\rvert^3} = -\frac{\mu\,\mathbf{d}}{\lvert\mathbf{d}\rvert^3}, \qquad \mu = GM.$$

This is the one-body Kepler problem with **gravitational parameter** μ. It is exactly solvable, which is why the panel can show real orbital elements rather than measured estimates. The individual positions come back from $\mathbf{d}$ and the barycenter by mass fraction:

$$\mathbf{r}_0 = \mathbf{R} - \frac{m_1}{M}\,\mathbf{d}, \tag{6}$$

$$\mathbf{r}_1 = \mathbf{R} + \frac{m_0}{M}\,\mathbf{d}. \tag{7}$$

Each body traces the *same conic*, scaled by the other body's mass fraction and rotated 180° from its partner. That is why the equal-mass binary shows two identical interlocking ellipses, while a star with a light planet shows a huge planetary ellipse and a barely-visible stellar wobble.

8 Orbital elements from the state vector

The module `src/orbit.rs` derives the full element set analytically from the instantaneous relative position $\mathbf{d}$ and velocity $\mathbf{v}$, with $r = \lvert\mathbf{d}\rvert$. No softening is applied here: these are the

elements of the ideal Kepler orbit.

$\xi = \dfrac{\lvert\mathbf{v}\rvert^2}{2} - \dfrac{\mu}{r}$	specific orbital energy
$h = \mathbf{d} \times \mathbf{v}$	specific angular momentum
$\mathbf{e} = \dfrac{(\lvert\mathbf{v}\rvert^2 - \mu/r)\mathbf{d} - (\mathbf{d}\cdot\mathbf{v})\,\mathbf{v}}{\mu}$	eccentricity vector
$e = \lvert\mathbf{e}\rvert$	eccentricity
$a = -\dfrac{\mu}{2\xi}$	semi-major axis
$r_p = a(1-e), \qquad r_a = a(1+e)$	periapsis, apoapsis
$T = 2\pi\sqrt{a^3/\mu}$	period (Kepler's third law)
$\omega = \operatorname{atan2}(e_y,\, e_x)$	argument of periapsis

The symbol ξ is used for specific orbital energy rather than the ϵ many texts use, because ε is already the softening length of Section 3.

8.1 The eccentricity vector

e is the **Laplace–Runge–Lenz** vector. It is a conserved vector unique to the inverse-square force: it points from the focus toward periapsis and its length is the eccentricity, so one expression yields both the shape and the orientation of the orbit.

8.2 Classification is by energy, not eccentricity

$$\xi < 0 \;\Rightarrow\; \text{elliptic (bound)}, \qquad \xi \approx 0 \;\Rightarrow\; \text{parabolic}, \qquad \xi > 0 \;\Rightarrow\; \text{hyperbolic (escape)}.$$

This distinction is not pedantic. A radial free-fall (preset 6) has $h = 0$, and **e** reduces to $-\hat{\mathbf{d}}$, so $e = 1$ exactly — eccentricity alone would call it parabolic. Its energy is plainly negative, and the energy test correctly reports a bound, degenerate ellipse with $r_p = 0$ and r_a equal to the drop distance. The panel shows exactly that: $e = 1.0000$, apoapsis 520.0, $L = 0$.

For the true parabolic case a diverges, so the periapsis falls back to $r_p = h^2/(2\mu)$, and apoapsis and period are reported as undefined.

9 Drawing the predicted orbit

The ghost path is the conic in polar form about the focus,

$$r(\nu) = \frac{p}{1 + e\cos\nu}, \qquad p = a\,(1 - e^2),$$

where ν is the true anomaly, measured from periapsis, and p is the semi-latus rectum. Sweeping ν symmetrically about zero and rotating by ω gives the curve; the sweep direction follows the sign of h so the drawn path runs the way the body actually travels. Each sampled point is then mapped to both bodies through the mass fractions (6) and (7).

Two degenerate cases are handled explicitly:

- **Hyperbola.** r diverges at the asymptote, where $\cos\nu = -1/e$. The sweep stops at $0.985\arccos(-1/e)$, just inside it, so the branch is drawn without shooting off to infinity.
- **Radial orbit.** $e = 1$ gives $p = a(1 - e^2) = 0$, and the function returns an empty path. This is correct: the trajectory is a line segment, not a conic with area, so nothing is drawn.

10 Building the initial conditions

Presets are specified the way an astronomer would state them — masses, semi-major axis, eccentricity — and `kepler_pair` converts that into state vectors. Bodies start at **apoapsis**, $r = a(1 + e)$, with the speed given by the **vis-viva** equation:

$$v^2 = \mu \left(\frac{2}{r} - \frac{1}{a} \right).$$

At apoapsis the velocity is perpendicular to the separation, so the relative state is $\mathbf{d} = (r, 0)$ and $\mathbf{v} = (0, v)$. Splitting about the barycenter, each body receives the *other* body's mass fraction:

$$\mathbf{r}_0 = -\frac{m_1}{M}\mathbf{d}, \quad \mathbf{v}_0 = -\frac{m_1}{M}\mathbf{v}, \qquad \mathbf{r}_1 = +\frac{m_0}{M}\mathbf{d}, \quad \mathbf{v}_1 = +\frac{m_0}{M}\mathbf{v},$$

which yields $\mathbf{P} = 0$ and $\mathbf{R} = 0$ by construction.

Worked example (preset 1)

With $m_0 = 1000$, $m_1 = 8$, $a = 220$, $e = 0$:

$$\begin{aligned} \mu &= 200 \times 1008 = 201\,600, \\ v &= \sqrt{\mu / a} = \sqrt{916.36} = 30.27 \text{ world units/s}, \\ T &= 2\pi \sqrt{220^3 / 201\,600} = 45.66 \text{ s}. \end{aligned}$$

The panel reads $T = 45.66\,\text{s}$ — the same closed form, so that agreement only confirms the setup. The stronger check is that the *integrated* orbit returns to its starting point after exactly that much simulated time, which is what the test `body_returns_to_start_after_one_period` asserts.

11 Units

Units are arbitrary but internally consistent, with $G = 200$ chosen so a typical orbit spans a few hundred world units and completes in tens of seconds — fast enough to watch, large enough to avoid `f32` rendering artifacts. Body draw radius follows constant density,

$$R = 2.6\, m^{1/3},$$

so mass differences read as size differences without a heavy body swamping the view.

12 Rendering

Drawing is a pure function of state; nothing in `src/render.rs` mutates the simulation.

Camera. World space is y-up and metric, screen space is y-down and in pixels, with z the zoom in pixels per world unit:

$$\begin{aligned} x_s &= \frac{W}{2} + (p_x - c_x)\, z, \\ y_s &= \frac{H}{2} - (p_y - c_y)\, z. \end{aligned}$$

Zooming at the cursor converts the cursor to world space before and after the zoom change and shifts the center by the difference, so the point under the mouse stays put. Camera tracking uses framerate-independent exponential smoothing,

$$c \;+\!=\; (\text{target} - c)\big(1 - e^{-12\,\Delta t}\big),$$

which keeps a fast orbit from jittering the view. `F` frames the whole predicted orbit — not just the current separation, which would let a light body swing off screen half a period later — and accounts for the width of the panels so the orbit is not centered underneath one.

Trails. Sampled by distance rather than by time: a point is appended once the body has moved 1.2 world units, capped at 3000 points. Sampling per frame would make trail length depend on frame rate and time scale. Alpha ramps as $(i/n)^{1.4}$ from tail to head.

Bodies. The glow is 14 concentric discs out to 2.6× the radius, each at alpha 0.028, summing to $1 - 0.972^{14} \approx 33\%$ opacity at the core. Many faint layers integrate into a smooth falloff; a few strong ones would band into visible rings.

Vectors. Velocity and acceleration arrows are each normalized to the larger of the two bodies, so the pair stays comparable at any zoom and never overruns the screen.

13 Source layout

File	Contents
`src/vec2.rs`	`f64` 2D vector
`src/physics.rs`	bodies, forces, the Verlet step, conserved quantities
`src/orbit.rs`	Keplerian elements and the predicted conic
`src/presets.rs`	starting configurations
`src/camera.rs`	world/screen mapping, pan, zoom, framing
`src/render.rs`	all drawing
`src/main.rs`	input, the fixed-step loop, wiring

14 Tests

```
cargo test --release
```

The suite checks the physics, not the rendering:

Test	Asserts
`circular_orbit_keeps_its_radius`	radius holds to 10^{-3} over 200 000 steps
`verlet_does_not_drain_energy`	drift $< 10^{-4}$ on an $e = 0.8$ orbit over 400 s
`body_returns_to_start_after_one_period`	position closes to 10^{-3} after exactly T
`barycenter_and_momentum_stay_put`	$\mathbf{P}$ and $\mathbf{R}$ remain zero
`elements_recover_the_construction_parameters`	derived e and a match the preset inputs
`elements_classify_an_unbound_orbit`	flyby reports hyperbolic, $e > 1$, no period
`softening_survives_a_head_on_collapse`	radial infall stays finite through contact

The period test is the strongest of these: it closes the loop between the integrator (Section 4) and the analytic elements (Section 8), which are computed by completely separate code. If either were wrong, they would disagree.

15 Building this document

15.1 Prerequisites

The document uses only standard packages — `amsmath`, `amssymb`, `geometry`, `graphicx`, `booktabs`, `listings`, `xcolor`, `hyperref`, `microtype` and `lmodern` — so any reasonably complete TEX installation will do.

Platform	Install
Debian / Ubuntu	`sudo apt install texlive-latex-recommended texlive-latex-extra texlive-fonts-recommended latexmk`
Fedora	`sudo dnf install texlive-scheme-medium latexmk`
Arch	`sudo pacman -S texlive-basic texlive-latexrecommended texlive-latexextra`
macOS	`brew install -cask mactex-no-gui`
Windows	MiKTeX (https://miktex.org) or TeX Live

A lighter alternative is **Tectonic**, a single self-contained binary that downloads only the packages a document actually needs:

```
cargo install tectonic          # or: brew install tectonic
```

15.2 Generating the PDF

From the `docs/` directory:

```
latexmk -pdf two-body.tex       # recommended: reruns as needed for the ToC
```

Equivalent alternatives:

```
tectonic two-body.tex           # one command, no latexmk needed

pdflatex two-body.tex           # plain pdflatex: run TWICE, so that the
pdflatex two-body.tex           # table of contents and cross-references resolve
```

All three write `two-body.pdf` beside the source. A `Makefile` is provided, so `make` in `docs/` does the same thing and `make clean` removes the intermediate `.aux`, `.log`, `.out` and `.toc` files.

The figure `elliptical.png` must be present in the same directory. It is committed alongside this source, and can be regenerated at any time with:

```
cargo run --release -- --capture docs/elliptical.png --preset 2
```